



CS3006- Parallel and Distributed Computing

Assignment 03

Maximum Marks :100

Submission Guidelines:

- The assignment Section A,B must be handwritten and Section C is Coding Part and Must implement in C/C++
- After completing the handwritten assignment, use a scanning app like CamScanner to create a digital file of your work and submit only this scanned file.
- Late submissions will not be accepted under any circumstances.
- Ensure that your work is original; any plagiarized content will result in a score of zero for the entire assignment.

Section A

Question 1 : Parallel Matrix Multiplication Engine

[15]

Scenario: You are deploying a parallelized matrix multiplication engine for large-scale data analysis in a scientific computing application. The best sequential algorithm for multiplying two $n \times n$ matrices takes 120 seconds to run (**TS = 120**). You decide to run this on a parallel architecture using 8 identical processing elements (**p = 8**). After implementation, you observe that the wall clock time (parallel runtime) is 20 seconds (**TP = 20**).

- Overhead Analysis:** Calculate the Total Parallel Overhead (**To**) for this system. Identify two specific sources of overhead from the slides that could cause the parallel system to not be 8 times faster than the serial version.
- Efficiency Metrics:** Determine the Speedup (**S**) and the Efficiency (**E**) of this matrix multiplication engine.
- Cost Evaluation:** What is the Cost of solving this problem on the 8-processor system? Is this system cost-optimal if the sequential complexity is **$O(n^3)$** and the parallel cost is also **$O(n^3)$** ? Justify your answer based on the asymptotic growth definition from the slides.

Question 2: Scaling the Graph Traversal Analyzer

[15]

Scenario: The graph traversal analyzer currently processes a graph with n vertices and edges. You are analyzing how the system behaves as it scales. You notice that as you keep the number of processors (p) constant, the efficiency (E) increases when you increase the graph size (n). However, if you increase p for a fixed graph size, the efficiency drops.

- a) **Iso-efficiency Metric:** Define what it means for this graph traversal analyzer to be a scalable parallel system in terms of keeping efficiency constant.
- b) **Problem Size (W):** If doubling the number of vertices/edges (n) results in a four-fold increase in execution time (like matrix addition), how should we precisely define the Problem Size (W) to ensure that doubling W always means doubling the computation? Base your answer on the definition given in the slides.
- c) **The "Wildcard" Extrapolation:** You have three algorithms for the analyzer: Binary Exchange, 2-D Transpose, and 3-D Transpose. If your current tests on a small machine (with small n) show Binary Exchange is fastest, why is it “difficult” to conclude it will be the best for the full-scale production environment with much larger graphs?

Question 3: Parallel Sorting with Granularity Adjustment

[15]

Scenario: You are implementing a parallel sorting module for a large database system. You are using a scaled-down version of a larger parallel system (fewer physical processors) to handle the sorting workload by assigning larger pieces of the input array to each processor.

- a) **Granularity Effects:** If you increase the granularity by assigning larger chunks of input data to each processor (scaling down from n processing elements to p processing elements where $p < n$), what is the maximum factor by which the overall communication time and the total parallel runtime will grow? (Assume appropriate mapping of virtual to physical processors.)
- b) **Cost and Optimality:** Explain the impact of this granularity increase on cost-optimality. If the original system with n processors was cost-optimal, what happens to cost-optimality after scaling down to p processors?
- c) **Scaled Speedup vs. Iso-efficiency:** If you increase the input size (problem size) linearly with the number of processors added, which scalability metric are you primarily measuring? If the resulting efficiency curve is sublinear (efficiency

decreases as p increases even with scaled problem size), what does that tell you about the system's scalability according to the slides?

Question 4 : The "Deep-Space" Signal Analysis

[15]

The Deep-Space Research Lab has captured a massive data transmission from a distant galaxy. To analyze the signal, they first ran a test on a **single specialized workstation** at the lab. It took the machine **3,000 minutes** to decode the first 10% of the signal. Recognizing that the full signal would take months to process, the team commissioned a **supercomputing cluster consisting of 150 high-performance processing elements (p)** to handle the job in parallel.

When the cluster finished, the logs showed that the **parallel runtime**—the time from the start until the final node completed its task—was exactly **25 minutes**. However, the scientists noticed the system didn't reach its "ideal" potential.

The system logs revealed three major issues:

1. The processors had to constantly exchange "packet headers" to synchronize their progress, a significant source of **inter-process interaction**.
2. Many nodes were caught in a "wait-state" because the data wasn't distributed evenly, causing **idling** while other nodes were still working.
3. Finally, the parallel algorithm they used required **extra verification steps** that the single workstation never had to perform.

Analysis Goal	Description / Interpretation	Formula & Calculated Solution
Baseline Workload	Time required by one workstation to complete the workload alone.	$T_S = 3,000$ minutes
Cluster Size		
Observed Finish Time		
Performance Gain (S)		

Fraction of Utilization (E)		
Processor-Time Product		
Total Parallel Overhead		
Performance Limitation		

Section B (MPI)

Question 1 : Project "Suhalat-Sync"

[20]

The development team for the “**Shadi4All**” platform is launching a distributed analytics engine to process wedding vendor data across a large cluster. You are the Lead Engineer responsible for writing the coordination logic.

The engine begins by spinning up a massive parallel environment where every worker has its own private memory space. To start, you must prepare the system to accept arguments from the command line. Once active, your first task is for every single worker to look at a "digital mirror" and identify exactly who they are in the group (their ID) and how many total workers are in the cluster. This is critical because the workload is split based on these numbers.

Before any work starts, there is a strict rule: the "**Global Checkpoint.**" No worker is allowed to start reading the database until every single node in the cluster has successfully checked in. After this checkpoint, the **Master Node (Rank 0)** reads a new configuration file containing a "Discount Percentage." This single value must be pushed out to every other node in the cluster so that all nodes apply the same discount.

As the processing continues, the Master Node realizes it needs to send a large, unique list of "**Target Vendor IDs**" to each worker. Instead of sending messages one by one, the Master takes a giant array and "deals" segments of it out to each worker like a deck of cards, so every node gets an equal, unique portion. Once the workers finish their calculations, they each hold a local "Sub-Total" of revenue. These individual values must be gathered and mathematically summed together, with the final total landing back at the Master Node for the CEO's report. Finally, to ensure no memory leaks or hung processes, the entire environment must be formally shut down and cleaned up.

Task from the "Suhalat-Sync"	ScenarioExact MPI Command	Parameters Mapped to Case Study	Technical Explanation
1. How do you start the system and handle the command line args?			
2. How does a worker find its "Digital Mirror" ID?			
3. How do you find the total size of the "Suhalat" cluster?			
4. How do you implement the "Global Checkpoint" rule?	MPI_Barrier(comm)	MPI_COMM_WORLD	It blocks until all processes have reached this point.
5. How is the "Discount Percentage" pushed to all nodes?			
6. How are "Vendor IDs" dealt like a deck of cards?			
7. How do you sum the "Sub-Totals" back to the Master?			
8. How do you perform the final "Clean up" of the system?			

Section C (MPI Code)

Question No 1	[10]
---------------	------

Hierarchical Tree-Based Broadcast and Reduction on a Large Cluster

Modern clusters often implement **hierarchical collective operations** to minimize latency and reduce load on root nodes. Instead of a flat broadcast or reduction, processes are

organized into a logical k -ary tree to perform multi-level communication and reduce hot spots. In this task, you will implement **tree-based broadcast and reduction** and compare them with naive (flat) implementations.

Requirements

Input

- Integer P (number of processes, $P \geq 8$).
- Integer k (tree fan-out, e.g., 2, 4, or 8).
- Root process r ($0 \leq r < P$).
- Each process initially holds a local array A_i of size N , filled with random values.

Parallel Execution

- Build a **logical k -ary tree** rooted at process r over all P processes.
- Implement:
 1. **Tree-based broadcast** of a vector B (size N) from root r to all processes.
 2. **Tree-based reduction** computing a global sum of all A_i into A_{sum} at root r .
- Also implement **naive versions**:
 - Broadcast: root sends directly to every process using point-to-point sends.
 - Reduction: every process sends its A_i directly to root.

Synchronization

- Ensure each tree level completes before the next level (barrier or explicit parent-children protocol).
- Avoid deadlocks by carefully ordering send/rcv at parents and children.
- Use tags or communicators to separate broadcast and reduction.

Performance Analysis

- Measure and report wall-clock time for:
 - Naive broadcast vs tree-based broadcast.
 - Naive reduction vs tree-based reduction.
- Vary P and k ($P = 8, 16, 32, \dots$ and $k = 2, 4, 8$).

Expected Output

- Correctness check: root's A_{sum} must equal the serial sum over all A_i .
- Print “**Tree Collectives: CORRECT**” or “**INCORRECT**” based on validation.
- A printed summary table of execution times for different P and k values.

Dynamic Deadlock-Avoiding Message Exchange in MPI

Modern MPI systems must handle unpredictable communication patterns while avoiding circular waits. In this task, you will design a deadlock-free multi-phase message exchange protocol between multiple MPI processes without relying on buffering.

Input

- Integer **P** (number of MPI processes), $P \geq 8$.
- Each process i maintains two arrays:
 - `send[i][j]` → message for process j
 - `recv[i][j]` → buffer for receiving messages

Execution

- Implement a **two-phase communication schedule**:
 1. **Phase 1**: Processes exchange messages with processes whose ranks differ by an odd number.
 2. **Phase 2**: Processes exchange with those differing by an even number.
- No process may perform a send unless it provably avoids deadlock.
- You must implement your own **dynamic ordering mechanism** using timestamps or partial ordering.

Synchronization Requirements

- Use **MPI_Barrier** between phases.
- Use custom **handshake messages** to ensure a send is safe before posting it.
- Track pending sends and enforce strict progress.

Performance Analysis

- Compare:
 - Pure blocking send/receive
 - Handshake-based safe send/receive

Expected Output

- Final `recv[][]` array printed by each process
- Deadlock-free completion
- A time comparison table printed to the terminal

Question No 3	[10]
---------------	------

Concurrent Distributed Priority Queue using Message Passing and Non-Blocking Operations

Distributed priority queues are fundamental in task schedulers and distributed systems. You will implement a **lock-free, message-driven priority queue** across several MPI processes.

Input

- P processes, each generating K random tasks with priorities
- Task format: (priority, payload)

Execution

- Use non-blocking send/recv
- Each process sends its tasks to the global root
- Root maintains a **binary heap**, but insertions arrive concurrently
- Implement **request-completion polling** using MPI_Test or MPI_Waitsome
- Apply **software backoff** to reduce communication congestion

Synchronization

- No global barrier allowed
- Local thread at root merges tasks concurrently
- Use status flags to guarantee safe buffer reuse

Performance Analysis

- Compare:
 - Blocking collection
 - Non-blocking with polling
 - Non-blocking with batching

Expected Output

- Final globally sorted priority list printed by the Root.
- Console output showing the throughput statistics for each method.
- Report on CPU utilization for the root process (busy-waiting vs. productive work).